Project AutoBot

Aditya Mohan, Jose l. Holgado, Aleks Gloukhman, Fangbo Shi, Marco Molinari, Oday Kabha, Vasilije Rakcevic, Andrej Savinov
Technische Universitat Berlin, DAI-Lab,
Ernst-Reuter-Platz 7, 10587 Berlin, Germany,

***Abstract*—There is huge development happening in the field of Autonomous Driving, and it is expected that its implementation can potentially reshape mobility by enhancing the safety, accessibility, and convenience of automotive transportation. One of the biggest challenges in Autonomous driving is dealing with uncertain environments where multiple variables, both internal states, and external inputs, need to be taken into account. This project presents an Autonomous Driving agent that can plan a safe trajectory to a pre-determined goal and drive through the lanes, taking moving obstacles, battery life, and external inputs such as traffic signs, into account as constraints, and finally park in a designated parking spot. Moreover, the agent is also able to maneuver to a battery-charging station when its battery life goes below a certain threshold. The solution has been implemented on a combination of ROS and MORSE environments, with a powerful approach to behavior modelling using State-Machines.**

***Keywords*—Autonomous cars, Self-driving cars, Traffic sign recognition, Lane following, Obstacle Avoidance, State Machines**

I. Introduction

The development of an Autonomous Vehicles is a highly experiment based collaborative task and requires teams to constantly integrate the individual work done on various models. SAE International standard J3016 [21] defines six levels of automation for classifying the sophistication of a system, starting at Level 0 of no autonomy and ending at Level 5 of high Autonomy, where the car is able to operate on any road and in any conditions a human driver could negotiate, and the input of human driver is at-most restricted to defining the destination. At this level, the car should be able to plan a safe trajectory through the possible various possible paths, subject to internal constraints imposed by the sub-systems of the vehicle and the external constraints imposed by the obstacles, traffic signs, lanes etc. Level-5 is also the level at which most of the advantages of Autonomous cars can be exploited.

One of the biggest advantages of Autonomous Vehicles is in the domain of Road safety: In 2014, there were 32,675 traffic-related deaths, 2.3 million injuries and 6.1 million reported collisions. Of these, an estimated 94% are attributed to driver errors with 31% legally intoxicated drivers and 10% are distracted drivers [25]. Fully Autonomous vehicles, and even ADAS systems, offer a solution where the accidents caused by the driver's negligence can be reduced while also offering a means of personal mobility that is more comfortable for the commuters. They also offer a means of mobility for people who cannot drive due to physical or visual disability.

This paper presents a system that allows an Autonomous Driving agent to navigate towards a user specific goal position, by planning a safe path in an uncertain environment consisting of different kinds of terrain and static and dynamic obstacles. The agent is able to observe and interpret the lane information while planning its trajectory and thus, tries to stay within the lane while moving towards the goal, and is also able to interpret the information conveyed by different traffic signs - turns, terrain information, stop signs and parking signs - and act on them accordingly. The system is also capable of monitoring its internal state and taking constraints imposed by the battery level into account while navigating. Thus, when the battery goes down the system is able to temporarily change the goal to the nearest charging station and move towards it for the purposes of charging the battery. Once the battery is charged, the system continues on its path towards the goal. In this case, the goal is the the parking spot, and so once the agent reaches the parking, it is also able to perform a parking maneuver and park itself in a proper manner.

The various sub-modules and the related algorithms have been developed under the ROS framework [23] and have been linked to a simulation provided by the MORSE engine [2]. Thus, the components are simulated in the MORSE engine and the agent, which has been developed in ROS, is able to read and write to these components through the mechanisms inherent in ROS.

This paper is structured as follows: In section III, a high level overview of the architecture is presented. The implementation of the algorithms and techniques used in the project are discussed in section IV. The evaluation parameters and the metrics used in the project are discussed in section V. Section VI concludes the achievements.

II. Related work

Autonomous driving has been one of the research fields that has grown faster in the recent years. Indeed, finding a full autonomous driving solution in real urban scenarios is quite complex an still an open challange. However, several notorious attempts have been made with remarkable improvements in particular fields, like SLAM and Object Detection. Indeed, breaking down the challenge of autonomous driving into smaller problems helps us in determining the different components and solutions that need to be complete and integrate together to obtain a full autonomous system. We can find

extensive studies in the literature such like [11], [6] and [30] where detailed descriptions of the components, methods and approaches that have to be included in a successful architecture of an Autonomous car are given. These examples were quite valuable for us to define our system specifications and build our architecture. Based on the literature and the lecture notes, we came out with 4 main topics that we had to work with and these were used as a based line to structure our tasks and objectives for this project: Navigation and control, Localization and Mapping, Environment perception and Decision Making. For each topic, the different Technical Groups (TGs) were created, and the members of each TG did a deep dive into the state of the art literature to find insightful ideas that can be applied to our problem. Furthermore, during the development phase, each group focused on finding solutions that solve the challenges in the most optimal way and can be later integrated to generate a single end-to-end solution that is capable of showing sophisticated autonomous driving in uncertain environments.

III. System Design and Architecture

Our project proposes a solution for autonomous car navigation and parking in partially known and observable environments with the presence of multiple mobile obstacles and pedestrians. To achieve such a solution, we have broken down the main challenge into four topics - Control Navigation, Self-localization and Mapping, Environment Perception, and Decision Making. Based on this structure and several iterations during the development process, we have defined an architecture that covers all the challenge requirements.

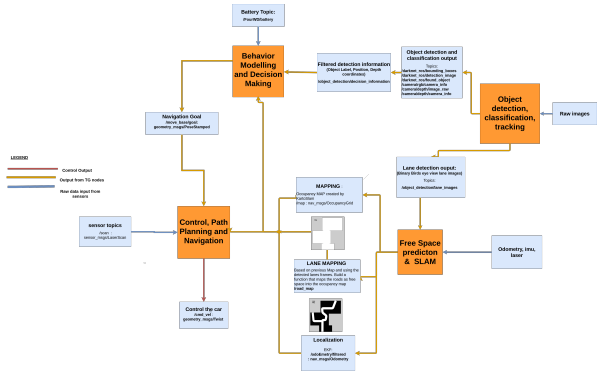

Fig. 1: Final Architecture and module Interfaces

The final architecture has 4 main modules and each of the modules take care of some part of the car's overall functionality, which are listed as follows:

- Control, Path Planning and Navigation
  - Planner
  - Speed and Steering control
  - Advance control strategies
- Behaviour Modelling and Decision making
  - Parking
  - Dynamic Obstacle avoidance
  - Navigation
  - Battery Management
- Object detection, Classification and Tracking
  - Object Classification and tracking
    - Obstacles and tracking
    - Traffic sign recognition and classification
    - Parking spot recognition
  - Lane Detection
- Free Space prediction SLAM
  - Localization using EKF
  - Mapping surrounding environment
  - Unsupervised Circuit tracks Free Space Prediction

Additionally in our architecture the final interfaces between the modules have also been listed. Each of the interfaces contains a brief description, name of the topics through which communication happens and the kind of message that is passed.

IV. Implementation Methodology

The system has been divided into four technical modules that handle end-end development of the various subsystems.

A. TG-1 Planner, PID and other improvements in control domain

Solution approach : The final approach was to make use of the planner and PID package by adjusting and configuring them to satisfy all control requirements. The major area of difficulty was trying to have stable control on transitions from high to low friction surfaces.

Architecture: The systems contains several control loops, as seen on Figure 2.

1) The planner from `move_base` stack has a control loop itself through the transform provided by simulation.
2) There is one more control loop located in TwistForce-Actuator: the P controller. Our team had a choice of either implementing PID in TwistForceActuator or at some higher level. Since a low level implementation, which we would need to write from scratch, would be quite error prone, and because partially working solutions for high level PIDs already exist, it was decided just to use the PID package provided by ROS and modify it to our needs. So we removed P controller from TwistForceActuator.
3) PIDs have a control loop through velocity feedback provided by the `Interface_PID` node. This node is used to receive and transform messages to PID form. `Velocity_Transform` node is responsible for applying transformation of velocities towards the `base_link` (car frame), as initially, `wheel_odom` provides velocities in global frame.

The Planner package required fine adjustments, especially keeping in mind the car geometry (for proper Ackerman steering). The PID package was modified to better deal with the integral windup.

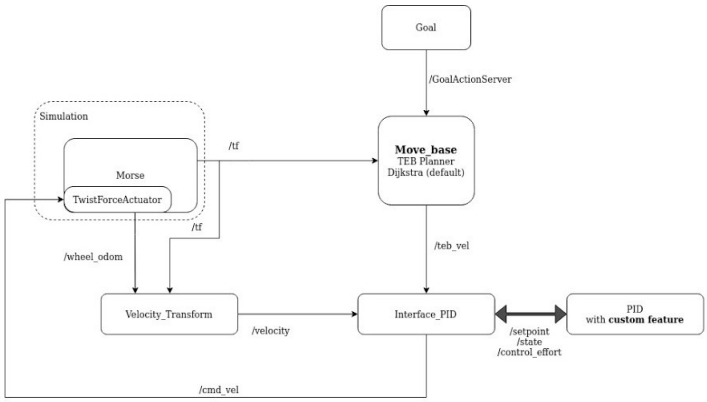

Fig. 2: System Architecture of TG1

Possible improvements: As presented in the System Design Specification document, our group also focused on experimentation with state of the art idea of including machine learning (ML) approaches for control. The idea was to replace the PID functionality with Neural Network (NN) which will provide more robust control, the inspiration for which can be found in [29]. A Neural Network can model arbitrary complexity depending on its configuration, and in that sense can be more powerful than just PIDs. Proper training and algorithm strategy is required for it to work. The working of this system can be seen in Figure 3. Results are presented in section V.

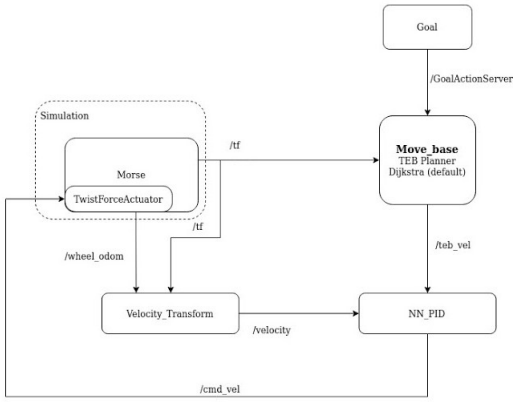

Fig. 3: Architecture for control with Neural Network -TG1

B. TG-2: Object Detection, Object Classification, Distance Measurements, Lane Detection

Solution approach : We examined the current state of research to find out which object recognition methods are the most efficient and common, and identified various solutions ranging from R-CNN [4, 3, 19], SqueezeDet [27], Detectron2 [28], YOLO [18] to Multi-View 3D [1]. Each of the proposed solutions was feasible for the project in terms of prediction accuracy. However, some of the solutions were not public or did not have an implementation available. Considering the short time-frame for completing our tasks, we decided to select those solution proposals which had an operational implementation. Additionally, we have preferred those implementations that have already been developed or modified for ROS. The underlying reasoning behind this was to save as much time as possible and create a prototype at an early stage.

After filtering out of all possible solutions, we ended up with Detectron2 [28] and YOLO [18]. We decided to use YOLO [18] to accomplish our desired goals. The reason for this was that a version of YOLO for ROS has already been developed [1] and YOLO is currently one of the most popular and best object recognition methods.

Object Detection : The available implementation follows a plug-and-play approach. However, we had to modify some configurations to guarantee fast and easy integration with other parts. This detection method consists mainly of three parts. The first one being a configuration file for the network, which contains details on how the model was trained and is used to initialize the model. The second part is a weight file, containing the trained weights for the model. The last part is a configuration file for the detection, which links to the path of the first and second part and contains all objects which can be detected.

To detect the specific objects of the simulation we had to create a new custom model. This model is responsible to detect all possible objects found while driving the car in the simulation. To create such a custom model it is necessary to modify an existing configuration file for the network. The modifications mainly describe how the network should be trained, its size, and how many objects will be classified and detected. Further on, we have to generate a new weight file by training a new model. We use YOLOs integrated functions to train our custom model by utilizing our network configuration file and pre-trained weights to fasten up the training.

The accuracy and precision of the custom model depend on the size of the network and the data set used for training. A bigger network size can result in an overall better object detection due to better precision and less loss. However, such a network requires a large amount of computational power. This limits our possible solutions to two different network sizes, a small model (i.e. 256x256) and a big model (i.e. 480x480). The size of a network is defined by its width and the height, where both have to be a multitude of 32 which is a requirement defined by YOLO.

To create a suitable data set we implemented an automatic image capturing solution which captures an image every 15 frames while driving the car manually. Furthermore, we have

[1] YOLO for ROS: https://github.com/leggedrobotics/darknet_ros

to label each image according to YOLOs requirements so that the training can recognize objects in the images. We used Yolo_mark [2] which is a labeling tool for YOLO. This tool offers a GUI to label images and creates the corresponding output needed for YOLO to train on images. Furthermore, we have divided the data set into training and validation sets. The former had about 3000 images and the latter about 150 images. Most images had small differences, which made it hard to select validation images that are unique, thus we only had a small amount for the validation.

The training was done on an Nvidia Geforce RTX 2070 SUPER and took about 10-15 hours, depending on the model. This GPU was also used for testing purposes and to measure the workload.

Object Classification : The bigger model seems to have higher precision and tends to handle the overall detection better than the smaller one. Furthermore, the smaller model had a slightly higher average loss than the bigger model, e.g. 0.11 compared to 0.8. However, the former requires less computational power and is capable of running at 60-80 FPS, while the latter one runs at 30-45 FPS.

Our solution is able to detect all objects within the simulation and classify them. Additionally, it is capable of displaying a bounding box overall detected objects. However, we had to increase the camera resolution due to YOLO not being able to create bounding boxes for small resolution images. Furthermore, we are capable of displaying the probability of classifications (i.e. precision) and showing the distance to detected objects.

Distance Measurements : To determine the distance towards detected objects, we modified an existing approach [3]. This solution uses the depth camera and the normal camera to estimate the distance between the car and the detected object. To determine the distance we use the coordinates of the bounding boxes of the detected objects. Thus, we take 9 different coordinates from within a bounding box and take the second smallest distance as our distance measurement. We use the second smallest distance as our measurement since the first smallest distance could be generated due to wrong image synchronizations.

The detected parameters are accessible from within the /darknet_ros/bounding_boxes topic.

Lane Detection : We did not use YOLO for lane detection, due to concerns that the parking spot detection and lane detection may interfere with each other. Thus, we modified a preexisting [4] computer vision approach to detect lanes.

This approach receives each image from the camera and applies one of two different solutions to detect possible lanes in the simulation. The first uses gradient thresholding and revolves around detecting the edges in an image. The second uses color thresholding to detect white colored objects in the simulation. We have observed that the color based approach produces a much clearer picture and contains very little if any noise. In contrast to that, the gradient-based approach was detecting edges of the other objects, did not fully display the lanes, and contained a lot of noise. To restrict the viewable area on the road we generated a trapezoidal region on each image. Further on, we transform the view angle to a bird's eye view perspective. The final images represent a binary bird's eye view image which is published to the /lane_images topic, which is used by TG-3 to generate a dynamic map.

C. TG-3: Free Space Prediction, Mapping and Localization

In this section, we are going to discuss the different approaches that we follow to solve the SLAM problem and how we used the lane detection to predict free space corridors for a save collision-free navigation.

Localization : The vehicle requires an accurate localization to navigate the road track. Therefore the vehicle is equipped with multiple sensors from which the vehicle localization can be estimated. The extended Kalman filter (EKF) is a common approach in robotics to use multiple sensors to get the best possible localization estimate. In this project, the implementation of the EKF by the robot localization ros package was used. For the sensor fusion sensor readings from wheel odometry, laser odometry and IMU were used. The configuration of the simulated sensors had to be reviewed and some sensor-related bugs were fixed before configuring the EKF. After configuring the EKF we were able to achieve accurate position tracking.

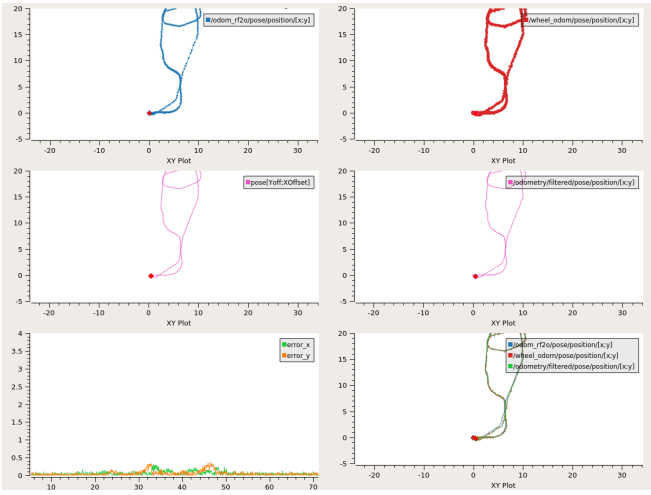

Fig. 4: Evaluation of position tracking in test environment

Mapping : One of the main challenges in robotics and particularly in autonomous driving is finding a precise representation from the surrounding environment that can be used later for free collision navigation. This is one of the core parts of the

[2] Yolo˙mark: https://github.com/AlexeyAB/Yolo_mark

[3] Distance Measurements in darknet˙ros https://github.com/AlejoDiaz49/Object-SearchLocalization-Robot

[4] Lane Detection: https://github.com/kenshiro-o/CarND-Advanced-Lane-Lines

SLAM problem. In this section, we are going to describe the benchmark that we have conducted to find out suitable for the box SLAM mapping algorithm that is tailored to our challenge specifications. Initially, we have been provided with hector slam [9]. Is a well-known algorithm that used lidar to generated a 2d map from the explore area. Although HectorSlam is a robust method for mapping in narrow spaces, we have discovered that isn't that good to map open spaces with few surfaces/Walls. Indeed, this is a known issue from Hector Slam. Additionally, we also have discovered that under fast maneuvers the map estimation harms the map quality. Considering these effects, we decided to explore other out of the box solutions available in ROS. We found 2 possible methods that were suitable for our case. The first one is GMAPPING, which is a Lidar SLAM technique that is based in the Particle filter and the second option is SlamKarto another Lidar-based solution that can employ the EKF-Odometry to localize the car within the generated map. After some benchmarking and research on the literature [10] we concluded that Karto-Slam produces the most precise map and have a bigger fit for our problem than the other two approaches.

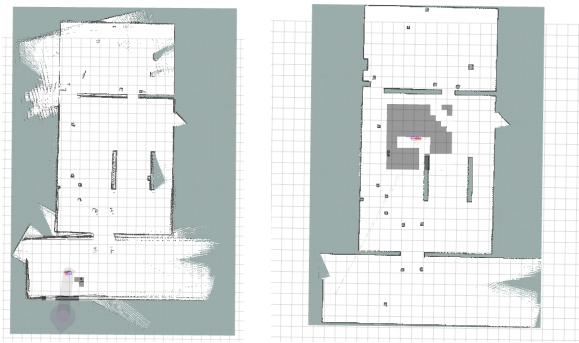

Fig. 5: Map comparison between HectorSlam (right) and KartoSlam (left)

Free space prediction: Understanding the explored environment and extracting relevant information from it is one of the most challenging tasks in autonomous systems. In our case, we were using the car position and the camera sensor to track lanes edge position respect to the car and predict the free available space between the car and lane edges. As an initial step, we proceed to localize the road edges in global coordinates. To accomplish this task, we firstly employed the car front camera and the depth camera. we marked the road detected edges by using the road detection algorithm provided by TG2. Them, we found out how far are the lanes edge points are from the car by using the Depth camera and the Pinhole model. This procedure provides us the road edge point cloud with respect to the car frame. After this step and knowing the car frame position, we can transform all the points from the point-cloud into the global frame.

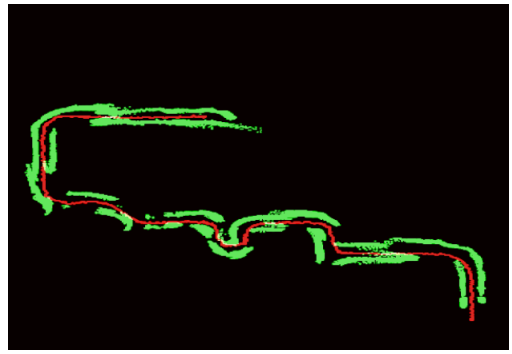

Fig. 6: Red: Car footprint, Green: Estimated lanes edges

After the points were transformed into world coordinates, we defined a polygon for each frame that limits the road's free-space between the car and the lane edges. To do so, we connected all edge points with the car footprint point, forming a close polygon that described the free space for this frame. Each polygon is stored in a list and later, by using a mapping tool, we were able to draw the free space corridor into an occupancy grid map, where the points with value 0 represent free space, and the points with value 100 represent occupied space. the output map of this node is published under the topic `/road_map` :. Later the topic was used to create the global cost map.

Fig. 7: Predicted Free Space Road Map

In general, we can observe that our predicted free space road map is reasonably good and similar to the lane ground truth. However, we have noticed that under certain circumstances the algorithm is over overshooting the road edges and that produces spikes in the final map.

We have found that our lane detection model understands bright reflections in walls as lanes edges. After several tests, we concluded that reflections can heavily impact the quality of the lane detection model for far points. A possible solution

for these issues might be to adjust the lane detection model parameters and make it more resilient against external light noises. Another possible solution might be, using the half-bottom region of the lane detection output image. By doing this, we can force the free space predictor to into account, only edge points that are close to the car.

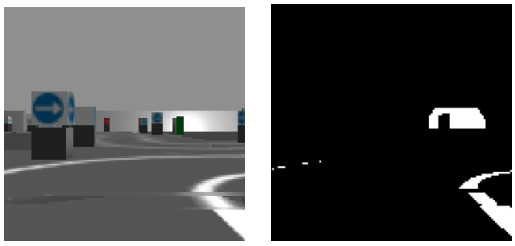

Fig. 8: Lane miss-detection

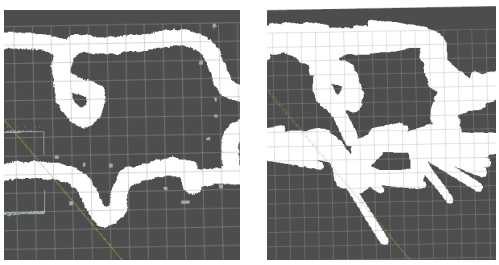

Fig. 9: spikes provoke by miss detection

D. TG-4: Behaviour Modelling, Battery Management and Obstacle Avoidance

Solution Approach: After having a route plan as a set of waypoints, the vehicle must coordinate several tasks to accomplish the desired goals. In the context of this project, the necessary functions include navigation based on road signs, dynamic obstacle avoidance, battery management, and parking. Each of these tasks has specific requirements and must comply with the traffic rules and restrictions such as, not crossing the track borders of the road. While the planner takes control of the latter part, the behavioral layer manages the other functions.

The approach followed in this project is to model the agent as a Discrete-Event System (DES) [17] which can be defined as an event-driven system where asynchronous discrete events trigger the transitions between states. Out of the several approaches [16] to achieve this modeling, two major ones are are Finite-State Machines [5] and Petri-nets [15].

We followed the Finite-State Machine approach. These are abstract machines with a set of pre-defined connected states, where the current state of the machine is selected by the transitions between states. Using this principle, the states can be connected in ways to build behaviors, which can then be further used to build more complex behaviors. The methods of composing behaviors are either using a hierarchical model to compose them or using a method to execute them parallel. Our reasons to use the State-Machine approach are twofold. First, it allows us to bring simplicity in the integration between the route planning node and the behavioral node, and second, the environment has a very definite structure where the unpredictable situations can be tackled by the motion planner, as opposed to the real-world stochastic situations.

Implementation : Using the input of the Perception and the Mapping nodes, the finite state machine should be able to cover the solution space, including recovery procedures and time/performance optimizations during the challenge. The Machine should also be able to directly interact with the sensors implemented in MORSE to aid it's decision process, specifically in the case of battery management. Finally, the state machine should be able to interact with the Planning node through goal points.

ROS contains several packages for modelling state dynamics [5]. One of the major approaches that exist in this field is FlexBE [22] which consists of a Graphical User Interface (GUI) that allows composing behaviors and states in parallel and hierarchical models. Appendix A further explains the working of the FlexBE software and how it is relevant to this project.

For the requirements of this project, the following containerized states were developed in FlexBE:

- **Move Base State:** This state is the encapsulation of the whole interaction of the behavioral layer with the planning node. It interacts with the planning node through `move_base/goal`. It sets up a simple action server [6] that sends a goal to the ROS navigation stack in the `map` frame and waits for the acknowledgment after the goal has been reached by car. The semantics of the goal sent are driven by the input to this state which is the userdata 'Direction'. The following navigation modes are specified:
  - `go_straight and go_back` : Move forward and backward by determining the current direction and according to a specified metric
  - `turn_left and turn_right` : Execute a turn by determining the turning direction and according to a turn metric
  - `Stop` : Stop the car by specifying the goal to be the same as the current pose
  - `Parking` : Be able to execute a parking maneuver by accepting a global parking goal
  - `Battery_in and Battery_out` : Be able to execute a maneuver of entering and exiting the charging station
- **Subscriber State :** This state subscribes to a ROS topic specified in the state's input and can pass on the received

[5] http://wiki.ros.org/decision_making/Tutorials/FSM

[6] http://wiki.ros.org/actionlib_tutorials/Tutorials/SimpleActionServer%28ExecuteCallbackMethod%29

message as the output. The topics states used in this project are:

- `/pose` : To read the current pose of the agent
- `/FourWD/battery` : To read the battery values
- `/darknet_ros/bounding_boxes` : To get the list of features detected by the perception node

- **Calculate_Need State :** This custom state has been designed to select the point at which the car should move into the parking mode. It takes the value in the topic `/pose` as an output of a subscriber state and checks if the pose is within a circle of a user-defined radius around the global paring co-ordinates. In case the car is within this region, the parking routine is entered to execute the parking maneuver.

- **Decide Maneuver :** This state is designed to switch between the type of movement the car has to execute. Based n the input from the `/darknet_ros/bounding_boxes` topic, as the output of a subscriber state, this state executes the following outputs:
  - `Obstacle`: If an obstacle is detected and it is within the range defined by a threshold, the state forces the automata to move into the obstacle avoidance routine
  - `Left/ Right`: If a navigation sign is detected within a threshold, then the state's output guides the automata to the respective maneuver routine
  - `none`: If no sign or obstacle satisfies the criterion, then the state's output asks the car to go straight for a certain distance and then check again

- **Avoidance_Check State:** This state is constructed for obstacle avoidance. It takes the pose as input and checks if the obstacle's distance is within a certain range. If the obstacle is closer than the lower threshold, then the state's output asks the car to move back a certain metric and then check again for the obstacle. If the value is inside the range, then the state asks the car to stop and observe the obstacle again. Finally, if the obstacle is farther away from the upper value, then the automata exit the obstacle avoidance routine

- **Wait State :** This state is entered at places to delay the automata for a certain time duration. It is primarily used for debugging purposes, or t stabilize transitions in case of erratic behavior

Using a composition of these states, behaviors are created to achieve functionalities. For this project, four major behaviors have been created:

Navigation: This behavior has the target of giving a navigation goal to the planning module. This navigation goal is given by taking into account the map and the objects detected by the perception node, and the battery and obstacle constraints explained further in the sections that follow. The goal is given through the `/move_base/goal` topic and consists of the

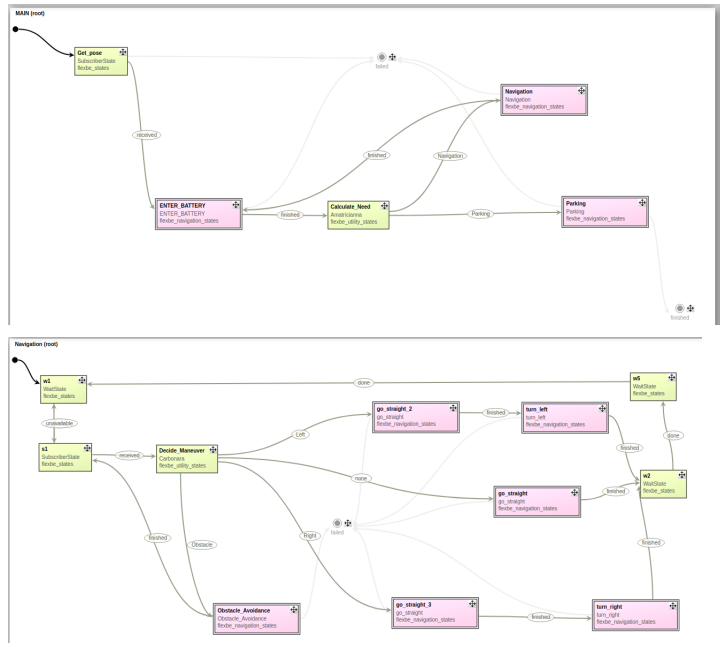

Fig. 10: The main behaviour that links all the functionalities (Top) and the Navigation behaviour that is the main sub-control loop

(x,y,z) position of the agent and the (x,y,z,w) pose of the agent at this position. The position that needs to be given is determined using the binary map from the planning node, which allows the goal to be set on the road and the pose is determined by comparing it with the current pose of the agent obtained by subscribing to the `/pose` topic and giving it in the form of increments to the location variables and transformations between the quaternions and Euler angles to the pose variables. As shown in figure 10 the decision here is calculated in priority order. The car first checks if any obstacles are encountered, and communicates with the planning node in case the car needs to stop or go back for immediate obstacles. If the car sees a turning, then the car updates the goal to a point in the new turning point. The case where two opposite turnings are present simultaneously is solved by the car comparing it's current position to the global goal and determining the range in which it needs to give the new pose, so that car constantly moves towards the goal.

Battery Management: The battery management behavior constantly monitors the `/FourWD/battery` topic and publishes the battery values. It also stores the global locations of all the charging points. At the point when the battery goes below 40%, a routine is triggered that finds the shortest Euclidean distance of a charging station from the current pose of the car and moves the car to that destination. Once the car reaches the charging station the navigation behavior passes the control back to the battery management behavior which performs an enter maneuver to the station ad keeps the car there till

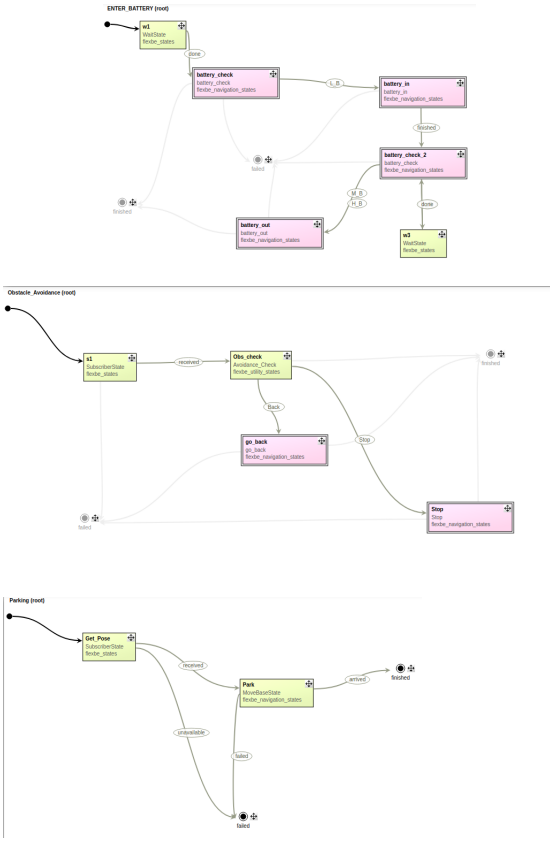

Fig. 11: Battery Management (Top), Obstacle avoidance (Middle) and Parking (Bottom)

the car can be charged. Once the battery reaches 100% the car is pulled back into its previous pose and the control is passed to the navigation behavior to drive the car to the global goal.

Dynamic Obstacle Avoidance: This behavior constantly monitors the output of the perception node and parses the received message for the Obstacle label. The distance of the nearest obstacle is extracted through a simple shortest element and is compared to a dynamic distance threshold range that is used to decide whether the car in its current pose can go forward with the approaching obstacle or if the car needs to stop or go back. In the latter cases, appropriate behaviors are activated.

Parking: The parking module specifically takes care of the parking maneuver. It monitors the input from the perception node and determines the occupancy of the parking. If a vacant parking spot is detected, then the mid-point of the parking spot is extracted and given to the navigation behavior to finally park the car. Once this maneuver is finished, then the whole scenario ends.

The composition of the listed behaviors allows the system to produce all the behaviors needed to cover the full set of requirements. The developed behaviors are all sequential, and their states block the flow execution until concluded. In conclusion, they can have several outputs so that different outcomes redirect to different actions as represented. Parallel behaviors are also supported by FlexBE but were not used for this project.

E. TG-X: Lane following

TG-X had the idea to implement a behavior-based lane following approach, that will work independent from vehicle localization and environment mapping. Some architectures with map-based navigation and behavior-based lane following that would work side by side were discussed.

A proof of concept based on an existing Github project[7] was implemented and presented.

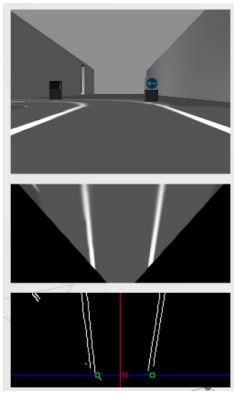

Fig. 12: TG-X lane following

As seen in figure 12, the camera image from the vehicle (top image) is transformed based on an Inverse Perspective Transformation. On the resulting picture (middle image) some image filtering and edge detection algorithms are applied to create the bottom image. Based on this image the control loop is steering the vehicle so it always stays between the detected lane markings.

V. Results

A. TG-1 Planner, PID and other improvements in control domain

General remarks: The control is stable and robust. The robot is able to navigate through all kinds of terrains and transition from one ground to another fairly well. This has been achieved through proper adjustments of the `move_base` package, especially the proper configuration of the TEB planner. PIDs are modified. Additionally, important control parameters were introduced and added to dynamic reconfiguration server for easy online adjustments. Simulation results can be obtained by running the code. Some of the interesting findings regarding control have been presented in the following paragraphs.

[7] https://github.com/StatueFungus/autonomous_driving

PID modification: In particular, a windup solution can be checked in Figure 13. In the left picture, it can be observed that the errors are slowly decreasing, which results in slow response times, as well as overshooting when making transitions from high to low friction surfaces. In the right picture, it can be observed that the integral part is being reduced proportionally. This results in fast reduction of errors, and with properly adjusted PD part of the controller, the car will be controllable in all situations and transitions.

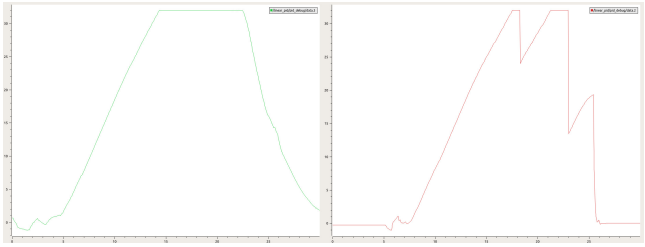

Fig. 13: Comparison of integral errors for PID without modifications (left) and with modifications (right)

Work on modes for easier integration: As a major obstacle in the project was to collaborate and try to integrate all the solutions from other groups properly, our group focused on ensuring easier testing of integrated solutions from other teams. In particular, the complete planner setup can be launched in 3 distinct ekf modes. The goal was to turn on and off ekf accordingly so that its work can be easily compared to launch with just bare sensor data coming from `wheel_odom`. We used this to verify performance similarities and differences. At the end our group worked mainly with noise turned off and with `wheel_odom` data, since ekf tends to over-smooth the incoming data. To further isolate dependence from other teams, the module for map creation was integrated and used in the beginning. Throughout the development phase, mostly static map was used, which was generated with that module. At the very end dynamic map creation solution was integrated into our branch.

Neural Network PID: The PID neural network (NN) has been written form scratch just using the pytorch library. While running this solution we noticed that even though robot was somehow responding to commands, overshoot was significant. The reason for that is the lack of proper training strategy for weight initialisation for this network. Even though it is a Reinforcement learning network, proper weight initialisation is crucial for performance. The idea is to reconfigure the NN so it just tries to imitate PIDs while running in parallel like presented on Figure 14, and then use these weights as initialisation weights for Reinforcement NN. While we couldn't complete the weight initialisation solution in this time-frame, this is quite good base to start with and we hope somebody would continue to work on that.

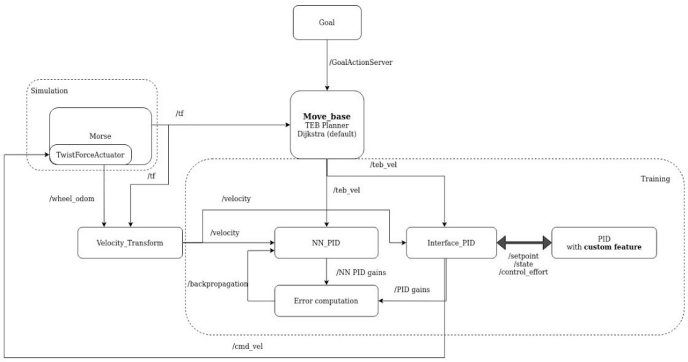

Fig. 14: Training strategy for Reinforcement Neural Network

B. TG-2 Object Detection, Object Classification, Distance Measurements, Lane Detection

General remarks: The proposed tasks and objectives were achieved with some small limitations. We did not create a solution that is capable of predicting the movement of objects. However, we were able to create a robust object detection pipeline, which detects every object in the simulation environment with high precision. The only flaw currently observable is that the left and right arrow signs can be mixed up from a further distance, i.e. a right arrow is classified as a left arrow and vice versa. The Distance measurements work fine until a certain distance to the object is reached, whereafter the measurement resets to 2.00 meters. This is due to limitations in the depth camera, where objects which are too close disappear from the camera (i.e. x-ray like behavior). Additionally, the camera resolutions for both the normal and depth camera had to be changed to 500 pixels, due to constraints from YOLO with lower resolutions. The lane detection is robust and works for all tested scenarios. However, when crossing speed bumps the camera loses track of the lanes, due to vertical changes. In such a case, the lane detection is not capable of detecting any lanes, as the camera can not see them.

C. TG-3 Free space detection, Mapping and Localization

General remarks. We proposed a SLAM solution based on EKF-KartoSlam that has achieved the main objective of precisely describing the car position with respect to the environment of the generated map. Regarding the mapping approach, we can observe a great improvement compared to the provided Hector-Slam solution, especially under edge circumstances such as fast steering maneuvers or navigation across a wide-open space. We can conclude that the resulting mapping strategy is robust enough to provide a robust 2D representation of the car surrounding environment.

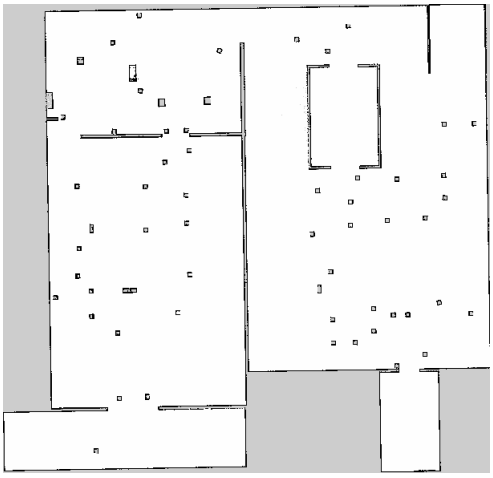

Fig. 15: KartoSlam Final Map

Furthermore, in the project, we also have introduced an unsupervised free space predictor method. In the following comparison, we can see that our algorithm can generate a reasonably good representation of the available free space between the car lanes. However, the algorithm takes a remarkable amount of computation power that doesn't make it suitable for embedded solutions. Some re-factorization on the code should reduce resource consumption. Additionally, Although our algorithm is capable of producing a reasonably good free-road map, there are still undesired spikes and noises present that impact the accuracy of our output map with respect to the ground-truth map. To reduce the effect of nuances, we suggest to consider only the closest lane landmarks points to the car to estimate the road free space and, explore new lane detection algorithms that are more robust to reflections.

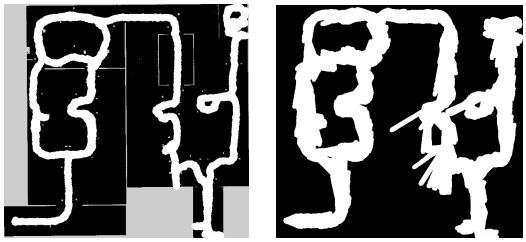

Fig. 16: Comparison of gt vs predicted free-space

Future Outlooks To improve the road free space prediction we can use robust deep learning algorithms for image segmentation. For this, using Generative adversarial networks for lane detection and free space prediction looks like a promising direction. We can find several studies in literature such like [12] where cGANS are used for image semantic segmentation with outstanding results. For our case, we could feed the network with images where lanes are present and the corresponding segmentation label.

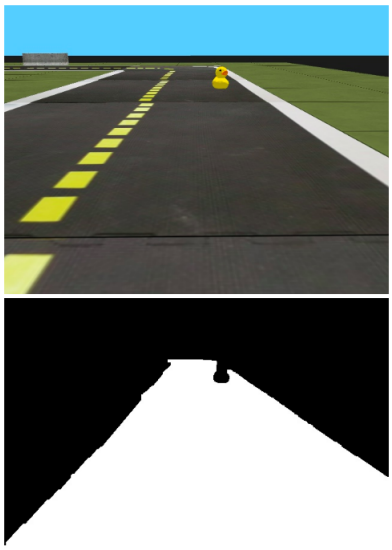

Fig. 17: example of road space segmentation

In addition to cGANs, we can find another interesting approach of road space segmentation based on deep learning that was created in the context of the DuckieTown project. This method combines FC-DenseNet, cGANs and Domain adaptation techniques to generate the road segmentation mask [20].

D. TG-4: Behaviour Modelling, Battery Management and Obstacle Avoidance

General Remarks: The major factors that are central to achieving full integration of movement and executing all the maneuvers are:

- `Lane Map` : This is important for extracting the turning locations with respect to the distance of the road signs, as viewed from the car frame. For this the car needs to know to be able stay in the lanes while executing the plans and needs to be able to interpolate turning directions and distances with respect to its current pose.
- `Origin Correction` : Since the goals are given in the map frame, the car needs to be able to factor-in the location of the origin in the world map, and successfully set goals that are within the range of the global cost-map
- `Tuned PID configurations`: This is linked to the movement in different terrains

The above requirements are inter-linked and an integrated goal-setting mechanism cannot be executed if even one of them does not work. Due to the working constraints, integration issues, the time-frame of the project and the individual turn-around-time for working modules, full-integration of the goal setting mechanism could not be achieved. However, the vehicle can execute the individual behaviors with user-defined parameters,which are listed as follows:

- Metrics for straight movement and turnings

- Origin correction based on the car's custom starting location
- Global goals in case of the battery and parking in terms of the starting position of the car

The execution of these unit tests has been shown in the submitted results, and can be listed down as follows:

- `Navigation:` With a defined straight and turn metric, the car can read the traffic signs, and use its pose to incrementally execute turnings for constant relative distances
- `Battery Management:` With a custom placement and global goal, the car can monitor its battery level, enter the charging station when it goes below the defined threshold, stay at the charging station till the battery is charged, and exit the charging station after the battery has been charged
- `Obstacle Avoidance` **:** With the defined threshold for detection, and custom placement, the car can stop on seeing an obstacle and based on it's relative position, even go back, after which it continues moving straight.
- `Parking:` With a custom placement, the car can execute a parking maneuver when given a global goal with respect to the car's frame.

Future Outlook: With the possibilities provided by the approach followed for behavior modelling and the completion of all the other nodes, the task that remains to be accomplished is integrated goal-based navigation for the the entire track .

With the various approaches that have been tried, the following future outlooks can be possible:

- Integration of the TG-X Lane following functionality as a separate mode of operation and a transition logic to shift between the lane following and goal-based navigation.
- Segregation of sign obstacle avoidance and battery management into parallel automata that can be executed with the navigation loop for batter control and separation of obstacle avoidance into an interrupt-based behaviour
- Globally defining the metrics and thresholds as hyperparameters that can be customized over the execution of the integrated behaviour and treating them as learnable parameters in a Deep Reinforcement learning problem with Generative Modelling, as followed in [8]

All these outlooks are withing the range of possible solution spaces and given enough time, can be worked upon with promising results.

VI. Conclusion

This project attempts to implement a fully autonomous driving model car in a MORSE environment with ROS as the operating system. The whole navigation of the car is based on recorded SLAM map, traffic lane, and traffic sign detection and recognition. The car is able to navigate through an area with the help of a camera, LIDAR sensor, and the map. It can successfully reach a desired goal in the map by tracking the traffic sign and lane. The map and the Localization are based on SLAM. Both are done using the Karto slam Library for ROS. Traffic sign and Lane Recognition is done with YOLO Network, which uses convolutional neural networks for feature extraction. The navigation part of the car is divided into two different Planners. One is the global planner to calculate the shortest path from an estimated starting point to a set goal. The calculation is based on all charted obstacles within the recorded map and the A* algorithm. The second planner is the local planner to find the best route at the current incidents. It will re-plan the local route if the LiDAR sensor will detect uncharted obstacles. It will also re-plan the path if it isn't possible to follow the path without ranking. The local planner generates the velocity commands too. The behavior of the car is modeled using Finite State Automata and the car is able to perform various maneuvers as hand-overs between the states and is able to navigate around static and dynamic obstacles using this model.

Acknowledgment

The completion of any project depends upon cooperation, coordination, and combined efforts of several sources of knowledge. The Authors are immensely thankful to Prof. Dr. Sahin Albayrak for the opportunity to work on a project of such nature. We are also thankful to Orhan Can Görür for setting up the track, the environment, and constantly guiding through the various phases of the project.

We are immensely grateful to Martin Berger for the constant guidance, regular meetings, and the overall support by being our mentor.

References

[1] Xiaozhi Chen et al. "Multi-view 3d object detection network for autonomous driving". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 1907–1915.

[2] G. Echeverria et al. "Modular open robots simulation engine: MORSE". In: *2011 IEEE International Conference on Robotics and Automation*. 2011, pp. 46–51.

[3] Ross Girshick. "Fast r-cnn". In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1440–1448.

[4] Ross Girshick et al. "Rich feature hierarchies for accurate object detection and semantic segmentation". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2014, pp. 580–587.

[5] John Hopcroft, Rajeev Motwani, and Jeffrey Ullman. *Introduction to automata theory, languages, and computation - international edition (2. ed)*. Jan. 2003. ISBN: 978-0-321-21029-6.

[6] K. Jo et al. "Development of Autonomous Car—Part II: A Case Study on the Implementation of an Autonomous Driving System Based on Distributed Architecture". In: *IEEE Transactions on Industrial Electronics* 62.8 (2015), pp. 5119–5132.

[7] Sören Kammel et al. "Team AnnieWAY's Autonomous System for the DARPA Urban Challenge 2007". In: *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Ed. by Martin Buehler, Karl Iagnemma, and Sanjiv Singh. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 359–391. ISBN: 978-3-642-03991-1. DOI: 10.1007/978-3-642-03991-1_9. URL: https://doi.org/10.1007/978-3-642-03991-1_9.

[8] Patrick Klose and Rudolf Mester. "Simulated Autonomous Driving in a Realistic Driving Environment using Deep Reinforcement Learning and a Deterministic Finite State Machine". In: Nov. 2018.

[9] S. Kohlbrecher et al. "A Flexible and Scalable SLAM System with Full 3D Motion Estimation". In: (Nov. 2011).

[10] Xuan Le et al. "Evaluation of Out-of-the-Box ROS 2D SLAMs for Autonomous Exploration of Unknown Indoor Environments: 11th International Conference, ICIRA 2018, Newcastle, NSW, Australia, August 9–11, 2018, Proceedings, Part II". In: (Aug. 2018), pp. 283–296. DOI: 10.1007/978-3-319-97589-4_24.

[11] J. Levinson et al. "Towards fully autonomous driving: Systems and algorithms". In: (2011), pp. 163–168.

[12] Pauline Luc et al. "Semantic Segmentation using Adversarial Networks". In: (2016). arXiv: 1611.08408 [cs.CV].

[13] Sebastian Meyer zu Borgsen. "Nonverbal Communication During Human-Robot Object Handover. Improving Predictability of Humanoid Robots by Gaze and Gestures in Close Interaction". In: (2020).

[14] Michael Montemerlo et al. "Junior: The Stanford Entry in the Urban Challenge". In: *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Ed. by Martin Buehler, Karl Iagnemma, and Sanjiv Singh. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 91–123. ISBN: 978-3-642-03991-1. DOI: 10.1007/978-3-642-03991-1_3. URL: https://doi.org/10.1007/978-3-642-03991-1_3.

[15] T. Murata. "Petri nets: Properties, analysis and applications". In: *Proceedings of the IEEE* 77.4 (1989), pp. 541–580.

[16] Magnus Olsson. "Behavior Trees for decision-making in Autonomous Driving". In: 2016.

[17] P. J. G. Ramadge and W. M. Wonham. "The control of discrete event systems". In: *Proceedings of the IEEE* 77.1 (1989), pp. 81–98.

[18] Joseph Redmon et al. "You Only Look Once: Unified, Real-Time Object Detection". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016). DOI: 10.1109/cvpr.2016.91. URL: http://dx.doi.org/10.1109/CVPR.2016.91.

[19] Shaoqing Ren et al. "Faster r-cnn: Towards real-time object detection with region proposal networks". In: *Advances in neural information processing systems*. 2015, pp. 91–99.

[20] "Right Lane segmentation in Duckietown environment". In: (). URL: https://github.com/timurlenk07/onlab_duckietown.

[21] SAE, ed. *Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles*. URL: https://www.sae.org/standards/content/j3016_201806/.

[22] P. Schillinger, S. Kohlbrecher, and O. von Stryk. "Human-robot collaborative high-level control with application to rescue robotics". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 2796–2802.

[23] Stanford Artificial Intelligence Laboratory et al. *Robotic Operating System*. Version ROS Melodic Morenia. May 23, 2018. URL: https://www.ros.org.

[24] Sebastian Thrun et al. "Stanley: The robot that won the DARPA Grand Challenge." In: *J. Field Robotics* 23 (Jan. 2006), pp. 661–692.

[25] NHTSA TRAFFIC SAFETY FACTS, ed. *2014 crash data key findings*. URL: https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/812219.

[26] Duy Tran et al. "A Human-Vehicle Collaborative Driving Framework for Driver Assistance". In: *IEEE Transactions on Intelligent Transportation Systems* PP (Dec. 2018), pp. 1–16. DOI: 10.1109/TITS.2018.2878027.

[27] Bichen Wu et al. "SqueezeDet: Unified, Small, Low Power Fully Convolutional Neural Networks for Real-Time Object Detection for Autonomous Driving". In: 2016.

[28] Yuxin Wu et al. *Detectron2*. https://github.com/facebookresearch/detectron2. 2019.

[29] Wei SUN Xue-song WANG Yu-hu CHENG. "A Proposal of Adaptive PID Controller Based on Reinforcement Learning". In: *Journal of China University of Mining and Technology* 17 (2007), pp. 40–44.

[30] E. Yurtsever et al. "A Survey of Autonomous Driving: Common Practices and Emerging Technologies". In: *IEEE Access* 8 (2020), pp. 58443–58469.

APPENDIX

A. FlexBE

FlexBE is a state machine editor that is implemented as a Graphical User Interface (GUI). It has been used in many places, like FSM models in DARPA challenge [14], [7], [24] and has also found in humanoid robot design [13]. It is based on a well known ROS library named SMACH [8], a robust library that allows creating complex behaviors as a connection of several states resulting in a hierarchical state machine, as shown in figure 18.

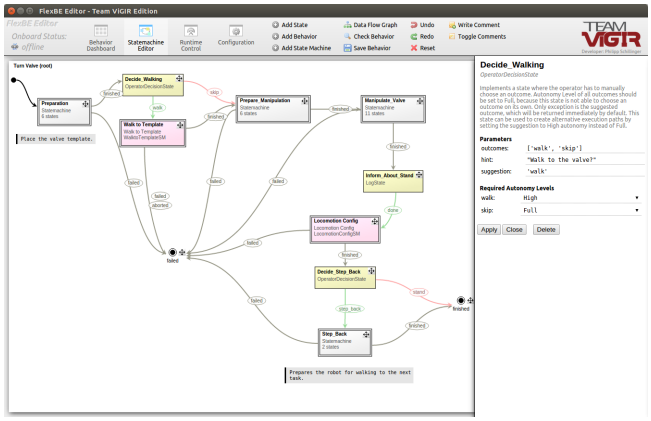

Fig. 18: Example of a FlexBE State composition from team vigir [9]. The boxes in yellow are base states, in pink composite behaviours and in white are containers

A significant advantage of this is the modularity of code, where the same functionality can be used in different parts of the task in hand, as well as an easy inspection of the state transitions and flows. It implements a drag and drop interface on its GUI, where one has a list of available states with different kinds of functionalities that can be connected. The SMACH hierarchical interface is implemented on FlexBE through joining one of several possible outputs of a state to the input of the next state and so forth. Moreover, these connected states are triggered by an initial point of execution and terminate in an ending state, which allows behaviors to be containerized as a state in higher-level behaviors. Each state works by event callbacks and follows a defined life-cycle which run pieces of code according to the status of the state, and FlexBE can run autonomously or monitored by an operator, who has live access to the execution status and can even receive a request for help in a certain tasks i.e Collaborative Autonomy [26] which, although not within the scope of this project, is a highly useful extension for real-world implementations before Level-5 Autonomy.

FlexBE is implemented over the Rosbridge, a JSON API that provides a ROS interface to other external software. Thus, it fits very well in a scenario where the agent reads from a physics engine environment - MORSE - and operates on a separate operating system - ROS. The communication with the other nodes happens through ROS topics to which the FlexBE behaviors can subscribe, or publish. The communication with the perception node happens by subscribing to the topic '/darknet' and entails receiving the messages of the object labels and their distances from the car. The communication with the mapping node happens over the topic '/karto-slam' which allows the planner to use the map for the goal-setting operation. The communication with the planning node happens through the topic '/move_base/goal', which is used to give the goals, which are used by the Teb-Local planner to plan the trajectory. All the states implemented in the FlexBE environment are python scripts that are, respectively, programmed automatically and by the user. Behaviors code is generated through the web-based user interface and dwells on state references and connections between them, whereas, states code are customized and programmed directly by the user to fulfill its needs according to the robot configurations.

The states are implemented in python and are defined as classes of their own, or inherited from other base classes. As shown in figure 19 the user interface, which is implemented in Javascript, interacts over Rosbridge with the backend which is implemented in python.

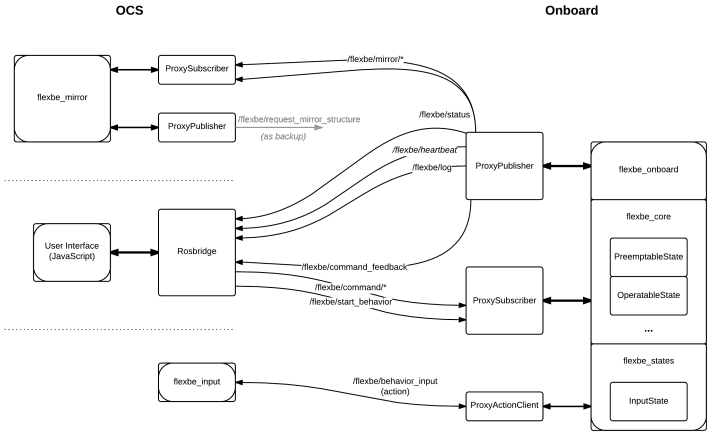

Fig. 19: FlexBE Communication Flow

Each behavior is written in Python and then interpreted into a C executable. The Linkage with the engine-core happens through Manifests, which are xml files that connect classes. The FlexBE core runs the ros master and publishes the functional messages from the `/flexbe_app` node, which connect them to the ros stack and the MORSE environment.

Thus, the software provides a layer of abstraction that allows a user to develop states without worrying about integration issues with the ROS environment. The states can be developed in blocking and non-blocking ways and can be made to preempt according to the needs of the user which can be customized according to application requirements.

[8] http://wiki.ros.org/smach